

Assignment 3 Basic Vulnerability Scanning Using Captured Packets

Contents

Introduction	1
Tools Used.....	2
Tools Used Exec Summary	2
Approach to Design.....	2
Approach to Design Exec Summary	3
Most Challenging	3
Most Challenging Exec Summary	3
Least Challenging	3
Least Challenging Exec Summary	4
Protocol and Port Combinations	4
Protocol and Port Combinations Exec Summary	4
Value of this Scanner.....	5
Value of this Scanner Exec Summary	5
How this relates to other scanning tools	5
How this relates to other scanning tools Exec Summary	6
Personal Enhancement	6
Personal Enhancement Exec Summary	7
Bibliography	7

Introduction

The focus of this exercise was to create a basic vulnerability scanner that inspects captured packet data and looks for known vulnerable protocol and port combinations. The

idea being that this software would sit on the border of a network and scan all incoming and outgoing network traffic looking for suspicious activity on ports and protocols that are known to be vulnerable.

Tools Used

To create this packet scanner, I wrote a python script that utilizes PyShark to inspect packet data. I chose Python and PyShark because they are both incredibly versatile, easy to use, and very feature rich for manipulating network packets. Both python and Pyshark can be used in almost any environment and have large amount of support and documentation. Next, in order to test this scanner, I needed network traffic packet captures which at first, I was generating myself and inspecting manually using Wireshark, however eventually I found better examples of network traffic available online here (Weber, 2025).

Tools Used Exec Summary

When it comes to writing software there are significant advantages to using well-known and established tools, such as Python and PyShark. The following article discusses many of these specifically for PyShark (PyShark: What It Is And How To Use, 2025). Python is very easily maintainable, well documented, and incredibly versatile given its cross-platform abilities. Adding a library like PyShark makes this combination nearly perfect for network traffic analysis / manipulation.

Approach to Design

To start on this exercise, I started with the skeleton code that was provided and expanded to add the features required. The general flow of the code was to loop over packets from a capture file and perform a series of if/else checks on each packet looking for known issues or vulnerabilities. Most of the time this involved extracting header information such as what type of protocol the packet was / is using then checking for things like the port being used and or the version being used; last checking if this combination is known to be safe vs unsafe. I did not run into many issues during the implementation of the script, the only difficult part was researching the various methods available in Pyshark and how to utilize them. Once I had a good understanding of how to use PyShark the actual python code itself was not significantly complicated. Again simply looping over packets and performing checks with if statements then tracking or saving into lists the various protocols and combinations that I found to be safe or unsafe.

Approach to Design Exec Summary

The actual mechanics behind packet scanning are relatively simple. All network traffic is transmitted in packets and the act of scanning network traffic is to take each of those packets and use software to extract specific information that tells a network engineer useful information about the traffic. In this case I used Pyshark to look at the protocols being used, the port used, and the version type in some situations. This information can be used to determine if there are vulnerabilities in that kind of network activity. The scanner I created does just that and then outputs all the collected information to a console.

Most Challenging

For me the most challenging part of this exercise was researching all the information on which ports / protocols to look for and knowing which combinations are considered safe or not. The process of gathering this information involved a lot of browsing web pages and reading about different network protocols. Including things like basic website sites that listed port and protocol combinations and more in depth articles about what types of protocols use plain text or no encryption on the data being transmitted.

Most Challenging Exec Summary

In software development and most things, the hardest part of the process is not the creation but the preparation for creation. In this case I had to do significant research into what makes network traffic vulnerable as well as common traits about network traffic that could be used to identify it. To be more specific, I had to understand what kind of ports specific protocols use and or which versions of protocols are known to have vulnerabilities. However, this kind of research is critical not only to creating network scanners but in protecting and maintaining networks in general.

Least Challenging

The easiest part of this exercise was the actual gathering and printing of all the information. I set up my script to push collected statistics into lists that I created, and the end of the script is just dumping all the information into the console, which sometimes does mean dumping a large amount of information. The large capture I found from (Weber, 2025) creates thousands of lines and ultimately, I had to print these to a PDF in order to read.

Least Challenging Exec Summary

In software development the act of outputting data is relatively straightforward, especially if the information is stored in easily accessible lists. In this exercise I followed the pattern of the skeleton code and created more lists which at the end of the script I simply dump all the information from these lists to a prompt. This approach is relatively easy and ensures that all data is output for analysis.

Protocol and Port Combinations

To identify traffic as safe or not the first thing I needed was to identify what kind of protocol it was using and the common ports associated with it. The following is dictionary I created of known protocols and port combinations that I was identifying.

```
{  
  22: "SSH",  
  80: "HTTP",  
  53: "DNS",  
  21: "FTP",  
  23: "TELNET",  
  25: "SMTP",  
  110: "POP3",  
  143: "IMAP",  
  443: "HTTPS",  
  3306: "MySQL"  
}
```

I created this list from the information that I gathered reading several web pages such as (Ibeakanma, 2025). As mentioned in the article, all of these are known to be vulnerable in some way to attackers. For example, FTP, TELNET, and HTTP, IMAP all sometimes transfer data in plain text making these protocols extremely vulnerable to attackers that can inspect any kind of network information. Other examples are that POP3, and SMTP are often used without any encryption, also making these very vulnerable protocols.

Protocol and Port Combinations Exec Summary

Regarding network traffic there are a few specific things a network engineer can look for to determine if traffic is safe or not. The main two properties are the protocol being used and the port it is trying to talk to. In this case I used a combination of the two to look for

suspicious activity, this combined with a third property, the version, can usually tell a network engineer if traffic is considered safe or not.

Value of this Scanner

A network scanning script such as the one I created here is useful for several reasons. First it can identify potential weak points quickly, it ensures ongoing protection with real-time monitoring, and enhances manual techniques, offering a balanced security strategy, all these are discussed here (Orszula, 2025). All these traits are specifically helpful to network engineers because they are a passive way to detect vulnerable activity on a network and indicate that those services need to be updated or replaced with more secure protocols. For example, the network engineer could force users to use SSH rather than TELNET or HTTPS rather than HTTP or updating SMTP to its latest and most secure version.

For a hacker this kind of tool is valuable because again it identifies vulnerabilities that they could then exploit to gain access to data coming out of the network or even to gather credentials which could be used to access said network. For example, looking for information that is often transmitted in plain text when protocols like FTP, TELNET, or HTTP are used.

Value of this Scanner Exec Summary

Scanners like these are extremely useful for both network defenders and attackers. A scanning tool like this is a way to continuously gather information about network traffic that can find vulnerabilities which again is very useful for both network engineers and hackers. For either perspective, understanding the hosts, services, and properties of a network are crucial to working with said network.

How this relates to other scanning tools

This type of scanner fits in perfectly with the tools that were explored in the previous exercise because it is another tool that can be used to identify vulnerabilities in a network. This tool is a slightly more targeted and automated approach than any of the tools we used previously because it is looking for specific protocol / port combinations which would enable someone to quickly and easily find a method for exploiting an already known vulnerability. This is desirable for both the network engineer and the attacker because it gives more specific information and does so in a passive yet automated way. This makes gathering the information quick and easy, while also providing specific information on the

vulnerabilities found. All of this is discussed in the previously mentioned article here (Orszula, 2025)

How this relates to other scanning tools Exec Summary

Gathering information about network activity can be extremely insightful to network engineers and network attackers alike. The scanning script I created here is another tool that can be used in conjunction with previously explored techniques to further increase ones understanding of a network. This scanner differs from previously mentioned techniques because it is looking for specific types of activity and also does so continuously. This means that an attacker or defender could use this information to better hone their approach to the network regardless of which perspective we are taking on. Additionally, this method is a more passive and continuous method of collecting data meaning it will be hard to detect and can collect large amounts of data over a continuous time period.

Personal Enhancement

The final part of this exercise was to come up with an enhancement of my own design. The enhancement that I chose to implement was to identify packets with clear text credentials or sensitive data. Many older internet protocols unfortunately send their data in plain text, making this information vulnerable/accessible to anyone on the network. (Joan, 2025). I will admit that accessing this network traffic requires one to already have access to said network, this is still an extreme vulnerability that should be addressed whenever possible. (Joan, 2025). To check for the presence of plain text information I checked packets for protocols known to transmit this kind of information over plain text such as FTP, TELNET, and HTTP. I then checked for the USER or PASS commands in the packet session data. This enhancement has two benefits it helps detect risky use of protocols that are transmitting sensitive data and alerts the engineer specifically if credentials are being exposed to the network. I ran this new feature against a known packet source, and it did indeed find credentials being exposed on a network.

```
Packet 45110 uses vulnerable TLS version: TLS 1.0
```

Cleartext Data:

```
Packet 25335 contains sensitive data: FTP USER: ftp1119456-nureintest
Packet 25338 contains sensitive data: FTP PASS: 98fDMjEHP6kV0D62WhK1
Packet 25402 contains sensitive data: FTP USER: ftp1119456-nureintest
Packet 25405 contains sensitive data: FTP PASS: 98fDMjEHP6kV0D62WhK1
Packet 25724 contains sensitive data: FTP USER: anonymous
Packet 25727 contains sensitive data: FTP PASS: User@
Packet 25740 contains sensitive data: FTP USER: ftp1119456-nureintest
Packet 25744 contains sensitive data: FTP PASS: 98fDMjEHP6kV0D62WhK1
Packet 25771 contains sensitive data: FTP USER: ftp1119456-nureintest
Packet 25775 contains sensitive data: FTP PASS: 98fDMjEHP6kV0D62WhK1
```

Personal Enhancement Exec Summary

Many older or simpler network protocols do not bother to encrypt or hide the data that they transmit. Two easy examples of this are FTP and TELNET (Joan, 2025). For network defenders and attackers alike this kind of information would stand out immediately and be incredibly important. For the network defender the leaking of that kind of information would be a disaster and something that they would want to mitigate or prevent entirely because the credentials could be used to further penetrate the computer system(s). For the network attackers this information is significant for basically the same reason, they could further compromise the overall network.

Bibliography

- Ibeakanma, C. (2025, 1 29). *The 8 Most Vulnerable Ports to Check When Pentesting*. Retrieved from MakeUseOf: <https://www.makeuseof.com/vulnerable-ports-check-when-pentesting/>
- Joan, B. (2025, 1 29). *Difference Between Ftp And Telnet*. Retrieved from DifferenceBetween: <https://www.differencebetween.net/technology/internet/difference-between-ftp-and-telnet/>
- Orszula, B. (2025, 1 29). *Automated Network Scanning and Assessments*. Retrieved from Intervision: <https://intervision.com/blog-automated-network-scanning-and-assessments/>

PyShark: What It Is And How To Use. (2025, 1 29). Retrieved from Celery-Q:
<https://celeryq.org/pyshark/>

Weber, J. (2025, 1 25). *The Ultimate PCAP*. Retrieved from Weberblog.net:
<https://weberblog.net/the-ultimate-pcap/>